NUS Orbital 2026

Milestone 02 – Prototype

M1 Poster Archive M1 Video Archive App Logs

Atlas (Team ID:7660)
Wang Qichen (A0337219B)
Feng Yi (A0322277E)

Proposed Level of Achievement:
Artemis

Contents

Deployment

The deployed web application is available at: orbital-artemis-armap-nus.onrender.com

Motivation

Navigating through the NUS campus can be difficult because complete geospatial and indoor facility data is not always publicly available or intuitive to use. Even after reaching the correct building, students may still need to locate lecture rooms, tutorial classrooms, study spaces, restrooms, lifts, printing services, and other facilities.

These navigation needs are closely tied to students' daily routines and schedules. Atlas is motivated by the need for a smarter, more context-aware campus assistant that supports both outdoor movement across NUS and indoor discovery within selected buildings.

Our Aim

Atlas aims to create a seamless campus assistant for NUS students. The project combines map-based navigation, facility discovery, schedule-aware guidance, and a daily assistant interface so that students can move through campus more confidently.

For Milestone 02, our goal is to turn the M1 technical foundation into a usable prototype centred on the answer students need immediately: *how do I get from where I am to the next relevant place?* The prototype therefore joins authentication, campus search, multimodal route planning, live transit context, schedule-aware assistance, and an experimental camera-based AR guidance view into one end-to-end flow. M3 will focus on reliability, user testing, indoor data coverage, and refinement rather than adding disconnected features.

Milestone 02 Requirement Traceability

The Orbital briefing defines M2 as the **prototype** milestone. The system should implement the most essential user-facing answers, perform testing from unit to system level, document the system, and submit a README, project log, poster, and video. Atlas is aiming for Artemis, so the report also targets the Artemis evidence expected by the briefing: at least nine sufficiently complex features, two-week sprint tracking, GitHub Projects and code review, CI/CD awareness, software design justification, multi-level testing, and approximately 30 or more pages of M2 documentation.

Orbital M2 Expectation	Atlas Evidence in This Report	Current Status
Essential prototype	Search a destination, choose a travel mode, obtain a route, inspect transit context, and open AR guidance from the selected route.	Implemented prototype
9+ complex features	Authentication, recovery, search, personalised suggestions, OTP routing, map visualisation, AR guidance, NUS bus, Singapore transit, schedule/facilities, and LLM assistant.	11 feature areas
Unit to system testing	Four automated Go unit tests, frontend production build, backend package test, and a documented end-to-end manual matrix.	Partial; frontend automation is a known gap
Software design evidence	Layered architecture, API boundaries, ER/sequence/activity diagrams, design decisions, trade-offs, and failure handling.	Documented
Planning and review	Git history shows feature branches and merged/reverted AR pull requests; tasks and progress are tracked through GitHub issues.	In use; sprint evidence should remain current
CI/CD	Reproducible npm, Go, Docker, Compose, Caddy, and Render steps are defined. No committed GitHub Actions workflow exists at the M2 cut-off.	Deployment exists; CI automation pending
M2 submission set	This README complements the separately submitted Project Log, Project Poster, and Project Video.	README completed here

User Stories

The M2 stories below are limited to workflows supported by the current prototype. Indoor turn-by-turn routing, verified accessibility paths, emergency data, and community map editing remain future scope and are therefore not presented as completed user stories.

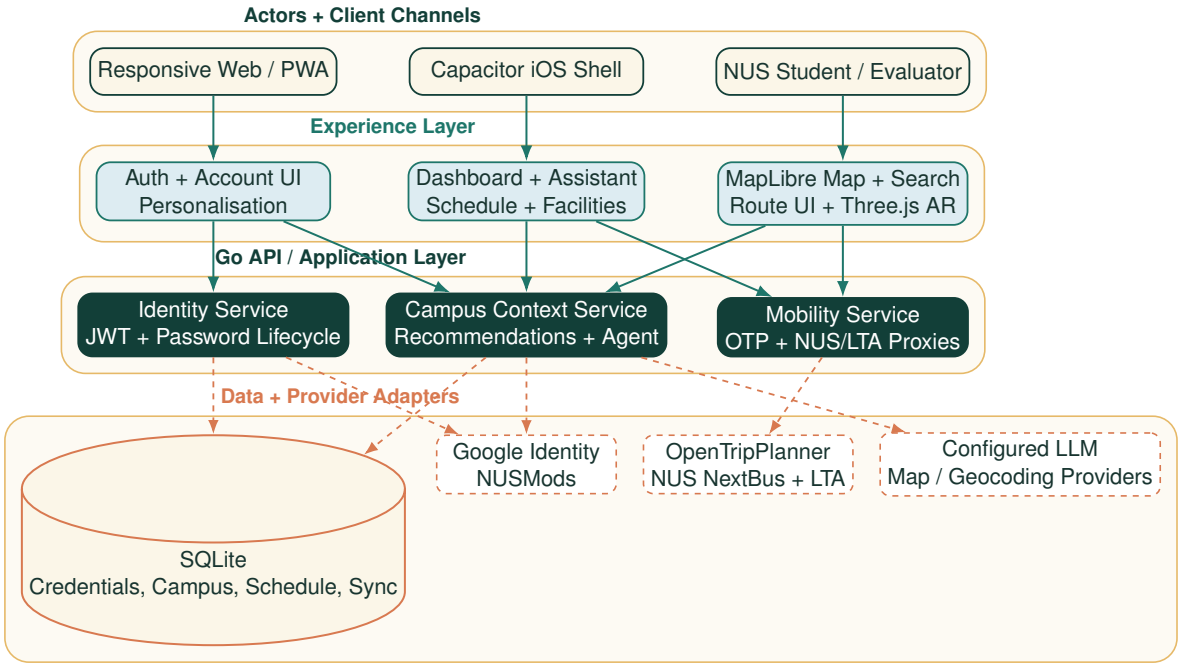
1. As an NUS student, I can register, sign in with email or Google, recover my password, and access protected features so that my campus data is not exposed through an anonymous session.
2. As a student opening Atlas before class, I can see my next event, time, location, map preview, and suggested actions on one dashboard so that I can decide what to do without visiting several pages.
3. As a freshman or exchange student, I can search common NUS abbreviations and wider Singapore destinations so that unfamiliar place names do not block navigation.
4. As a returning user, I can receive destination suggestions based on recent selections, visit frequency, time, and place tags so that repeated searches require fewer steps.

5. As a student planning a journey, I can choose walking, transit, driving, or cycling and compare route distance, duration, source, and alternatives so that I can select an appropriate travel mode.
6. As a student following a route, I can view origin and destination markers, mode-coloured route segments, live GPS position, and route summaries on the 2D map so that the journey remains understandable at a glance.
7. As a mobile user, I can open the selected route in an experimental AR view with heading, progress, next manoeuvre, remaining distance, and off-route feedback so that guidance can remain visible while I move.
8. As a student using NUS shuttle buses, I can view shuttle stops, routes, pickup points, estimated arrivals, active vehicles, alerts, and map overlays so that I can decide whether to walk or wait.
9. As a student travelling beyond campus, I can inspect nearby Singapore bus stops, bus arrivals, and rail service alerts so that the same application supports the wider journey.
10. As a student organising my day, I can create and delete schedule items, filter facilities by building and type, and inspect data-sync status so that navigation is connected to my immediate campus context.
11. As a student needing planning help, I can ask the Daily Assistant for a plan, task summary, or reflection and still receive a deterministic fallback if the configured LLM is unavailable.
12. As a student with different working habits, I can choose an avatar, theme, dashboard density, default page, study style, and route preference so that Atlas opens in a form suited to my routine.
13. As an evaluator or user in a restricted environment, I can see whether transit or assistant data is live, configured, fallback, or unavailable so that the prototype does not misrepresent generated or demo information.

Overall Architecture Diagram

Atlas uses a **four-layer client-server architecture**. The client channel layer supports the responsive web application and Capacitor iOS shell. The experience layer contains authentication, dashboard, map/search/routing, and AR views. The Go application layer provides a single security and orchestration boundary for accounts, campus context, route planning, transit, and the Daily Assistant. The infrastructure layer contains SQLite and adapters for Google Identity, OpenTripPlanner, NUSMods, NUS NextBus, LTA DataMall, and the configured LLM provider.

The main design decision is that browser components never call credential-bearing transit, identity-verification, routing, or LLM services directly. They call stable Atlas endpoints instead. The Go adapters translate provider-specific schemas, apply timeouts and normalisation, protect secrets, and label fallbacks. This creates more modules than a direct browser integration, but it prevents external API changes from spreading through the user interface and makes provider failures independently testable.



Atlas M2 layered architecture: solid teal arrows show application flow; dashed coral arrows show persistence or provider integration

Core Features

The current prototype is organised into 11 user-facing feature groups. Each group maps to at least one M2 user story and is backed by a concrete screen, API endpoint, persistent model, or testable service. Engineering infrastructure such as the monorepo, Docker files, and documentation is discussed separately and is not counted as a product feature.

S/N	Current Feature Group	Maturity	Implemented M2 Behaviour
Core Features			
1	Account and protected access	Working prototype	Registration, email/password and Google sign-in, signed JWT protection, security-question recovery, and authenticated password change form one account lifecycle.
2	Personalised home dashboard	Working prototype	The dashboard combines the next event, schedule preview, map preview, facilities, recommendations, assistant actions, service status, and navigation shortcuts.
3	Smart place search and ranking	Working prototype	Local NUS/Singapore aliases are merged with external geocoding; history, recency, frequency, time, and tags influence destination suggestions.
4	Multimodal route planning	Working with provider dependency	Users choose walk, transit, drive, or bike, request OTP itineraries, compare alternatives, and inspect source, distance, duration, and segment modes.
5	Interactive 2D navigation map	Working prototype	MapLibre renders campus markers, GPS location, start/end markers, mode-coloured routes, NUS overlays, route summaries, and selectable alternatives.
6	Experimental AR route guidance	M2 experimental	The selected map route is reused in a Three.js camera overlay with compass heading, GPS progress, next manoeuvre, remaining distance, calibration, and off-route feedback.
7	NUS shuttle integration	Working with live/demo sources	Atlas exposes stops, routes, pickup points, arrivals, active vehicles, alerts, nearby context, and route overlays through a server-side proxy.
8	Singapore transit context	Credential-dependent prototype	LTA adapters provide bus stops, nearby stops, arrivals, and rail service alerts while keeping the DataMall key outside the browser.
9	Schedule, facilities, and campus sync	Working prototype	Users can create/delete schedule items, filter seeded facilities by building/type, view deterministic recommendations, and inspect or trigger NUSMods sync status.
10	Daily Assistant	Working with tested fallback	Daily-plan, task-summary, and reflection modes accept app context, parse structured schedule drafts, and return deterministic fallbacks when the LLM is unavailable.
11	Personalisation and mobile delivery	Working prototype	Users choose avatar, theme, density, default page, study style, and route preference; responsive layouts and a Capacitor shell reuse the same web codebase on mobile.

The 11 groups deliberately avoid counting individual endpoints as separate features. For example, NUS bus stops, arrivals, and active vehicles are one coherent shuttle capability, while sign-in and recovery are one account lifecycle. This keeps the Artemis feature count tied to user value rather than inflated implementation details.

Feature Ideation

The ideation below records the user problem and the M2 engineering answer. The emphasis is on **why** each boundary was chosen, because naming a library without explaining the design decision does not demonstrate software engineering reasoning.

S/N	Feature	User Problem / Ideation	M2 Engineering Direction and Rationale
1	Authentication and account lifecycle	Personal schedules and assistant context require identity and a recoverable account.	Keep credential checks, Google token verification, password recovery, hashing, and JWT signing on the server. This prevents trust-sensitive decisions from being delegated to browser code and lets every protected feature reuse one middleware boundary.
2	Search and recommendations	Users should find both NUS abbreviations and wider Singapore destinations even when an external geocoder is slow.	Merge a curated local index with external results, and retain bounded search history in browser storage. Local-first results reduce latency and failure impact; ranking by context improves repeated use without transmitting search history to the server.
3	Multimodal routing	A straight line is not a usable answer for walking, cycling, driving, or transit.	Put OpenTripPlanner behind a protected Go adapter and normalise its GraphQL response into Atlas itineraries and segments. The UI consumes one stable shape while the backend owns timeout, schema, and fallback handling.
4	Map route visualisation	Students need to compare alternatives and understand mode changes at a glance.	Centralise source/layer mutation in `MapEngine`; keep the page responsible for state and selection. This avoids scattering MapLibre lifecycle calls across UI components and makes route redraw/clear operations deterministic.
5	AR route guidance	The key project hypothesis is that directional guidance can be more glanceable than repeatedly reading a 2D map.	Reuse the selected route instead of requesting a second route. Convert longitude/latitude to a local metre frame, snap GPS to the closest segment, and render only a route window in Three.js. This keeps the AR prototype consistent with the map and bounds scene complexity.

6	Transit integration	Campus travel often involves NUS shuttle buses and, outside campus, public buses or trains.	Proxy NUS and LTA providers through Go adapters, normalise provider-specific payloads, cache where appropriate, and label fallback/demo data. Credentials remain server-side and the UI can show source status honestly.
7	Daily context	A route is more useful when linked to the student's next class and nearby facilities.	Keep schedules, buildings, facilities, and sync records in SQLite behind API endpoints. Normalised records can be reused by deterministic recommendations and the assistant without duplicating state in the frontend.
8	LLM assistant	Students may ask for plans or summaries in natural language, but provider outages must not break the dashboard.	Use a small provider interface, build explicit system/mode/context messages, parse optional structured schedule drafts, and return deterministic mode-specific fallbacks. The adapter permits provider replacement and makes failure behaviour unit-testable.
9	Mobile delivery	Navigation is used while moving, so desktop-only success would not validate the product idea.	Use responsive React/React Native Web primitives and a Capacitor shell, while retaining one shared web codebase. This reduces duplicated feature work for a two-person team, though native sensors and permissions still require device testing.

Repository and Project Foundation

This section is not counted as a user-facing feature. It documents the engineering foundation that allows the team to build the user-facing features in a maintainable way: reproducible setup instructions, a clear project structure, issue tracking, and deployment configuration.

Project Structure and Version Control

Atlas is organized into a frontend, backend, deployment configuration, and supporting documentation. This mirrors the modular design described in the proposal and keeps responsibility boundaries clear. The frontend owns user interaction, the backend owns authentication and campus data APIs, and the deployment files define how the system runs in production.

Atlas uses a monorepo because the frontend, backend, deployment files, poster, and milestone report are tightly connected during M1. For a small two-person team, a monorepo reduces coordination overhead: one issue or pull request can show the full change across UI, API, documentation, and deployment. The trade-off is that the repository contains several types of work in one place, but this is acceptable because the project is still small and the folder boundaries are explicit.

```
Orbital_Artemis_ArMap_Nus/
|-- frontend/ React + Vite frontend and mobile app shell
| |-- src/ Auth, dashboard, campus map, and UI code
| `-- ios/ Capacitor iOS project
|-- backend/ Go API server
| |-- cmd/server/ Server entry point
| `-- internal/atlas/ Auth, campus data, routing, and API logic
|-- deploy/ Production deployment configuration
|-- poster_work/ Poster and presentation assets
|-- Dockerfile Container build definition
|-- docker-compose.yml Local multi-service run configuration
`-- readme.tex Milestone report source
```

Project Structure

The repository uses GitHub issues and pull requests as progress logs. Issue #1 initialized the repository and documentation baseline, while later issues track authentication, map rendering, search, dashboard integration, and mobile usability. This supports the proposal's branch-based workflow: tasks are decomposed into issues, implemented on feature branches, reviewed through pull requests, and eventually integrated into `main`.

The main trade-off of this structure is that it creates more files and coordination overhead than a single-folder prototype. However, the benefit is significant for an Artemis-level project: each module can evolve independently, deployment can be reproduced, and later AR/navigation extensions can be added without rewriting the whole codebase.

Project-Level Software Engineering

Atlas applies software engineering practices at the project level, not only inside individual files. The goal is to make the project trackable, reviewable, testable, and deployable throughout Orbital.

GitHub Project Management

GitHub issues are used as the team progress log. Each issue represents a concrete feature, defect, or engineering task, such as authentication, protected routing, map rendering, mobile layout, dashboard integration, and map polygon fixes. The public issue tracker functions as both a task board and an audit trail for mentor review.

The working workflow is:

1. Break milestone requirements into GitHub issues with clear acceptance criteria.
2. Assign work to a team member and connect it to the relevant feature area.
3. Implement changes on a focused branch instead of mixing unrelated work.
4. Review the change through a pull request before merging to `main`.
5. Use the issue and pull request history as progress logs for milestone submission.

This workflow is intentionally heavier than a quick prototype workflow. It is justified because Artemis projects are expected to demonstrate consistent engineering discipline and maintainability, not only visible screens. The AR history provides a useful example: an AR pull request was merged, judged not ready, and reverted through a second pull request. Preserving this decision in history is safer and more reviewable than rewriting the main branch.

GitHub Projects tab used to track milestone tasks, statuses, and review progress

M2 Sprint Plan and Allocation

M2 work was grouped into two-week objectives, with smaller issues and commits used for day-to-day tracking. The table records the intended outcome and the repository evidence at the reporting cut-off; detailed hour-by-hour contribution remains in the Project Log.

Sprint	Objective	Primary Allocation	Evidence / Outcome
3 (2–15 Jun)	Connect the M1 map to real transport and daily-assistant flows.	Wang: bus/agent/backend; Feng: GPS and mobile navigation experiments.	NUS bus map, GPS commits, assistant communication/actions, React Native shell, Docker and OTP integration.
4 (16–29 Jun)	Deliver route alternatives and an evaluable AR guidance prototype; stabilise documentation and tests.	Wang: search, OTP/display routing, agent tests; Feng: AR implementation and device-facing iteration; both: review and evaluation assets.	Campus search, route planning, AR iterations, PR merge/revert history, Go unit tests, M2 README.

The allocation describes responsibility areas, not a claim that every commit was individually authored in isolation. Cross-cutting integration and review are shared work. Contribution fairness should be evaluated together with the Project Log, issue assignments, pull-request discussion, and commit history rather than raw commit count alone.

Engineering Principles

Principle	How Atlas Applies It
Separation of concerns	Frontend pages and UI components are separated from backend API handlers, authentication logic, storage logic, and deployment configuration.
API-first integration	Frontend features call documented backend endpoints such as `/api/buildings`, `/api/facilities`, `/api/schedule`, `/api/recommendations`, and bus endpoints.
Progressive delivery	The team implements a reliable 2D map and schedule-aware dashboard before adding higher-risk AR and indoor navigation layers.
Security by default	Protected pages require authentication, backend data routes are guarded with JWT middleware, and Google ID token verification happens server-side.
Reproducibility	Local setup, environment variables, Docker build, Docker Compose deployment, and production Caddy configuration are documented in the repository.
Graceful degradation	Demo credentials, seeded data, and bus demo fallback data allow reviewers to evaluate the project even without private external API keys.

Risk Management

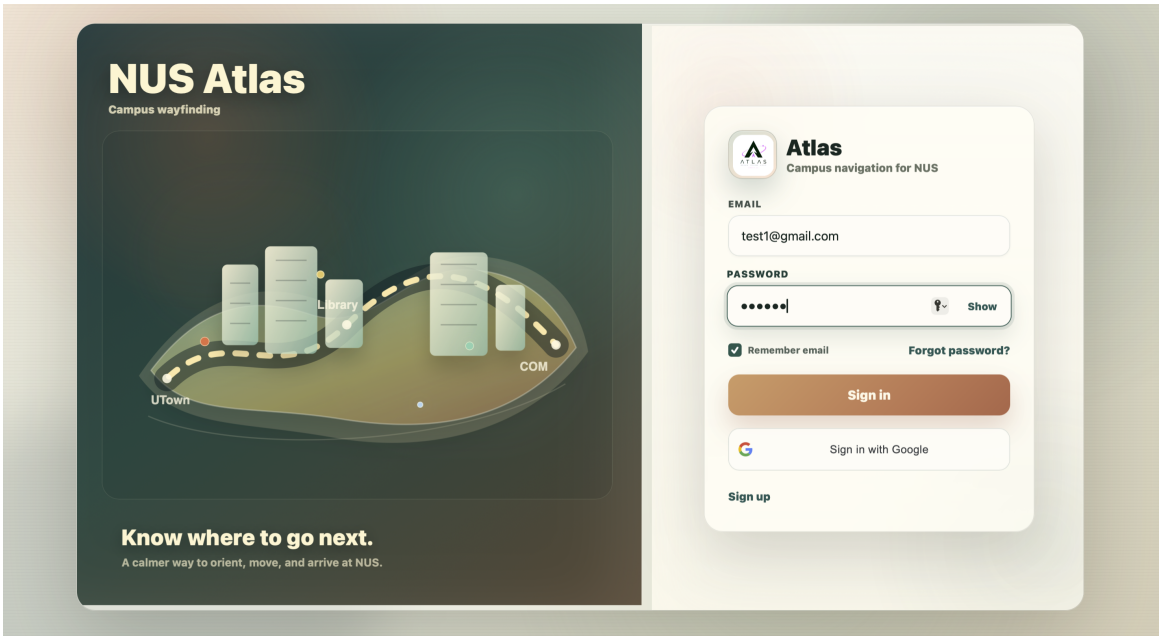
Risk	Impact	Mitigation
Incomplete campus/indoor data	The assistant does not yet cover every building, floor, or accessible path.	Use seeded demo data for core flows, design APIs around building/facility models, and expand verified coverage in M3.
External API reliability	NUSMods, Google verification, map tiles, or bus APIs may fail or require credentials.	Keep configuration in environment variables and provide demo/fallback behavior where possible.
AR sensor error	GPS and compass uncertainty can make the overlay misleading on narrow campus paths.	Display sensor/off-route status, reuse the 2D route, provide calibration, and treat M2 AR as an experimental prototype pending real-device user testing.
Mobile usability defects	Navigation apps are frequently used on phones, where cramped layouts can break flows.	Track responsive layout as a feature, include mobile screenshots, and test common auth/dashboard/map flows on small viewports.

Authentication and Protected Access

Authentication allows students to enter Atlas through email/password login, registration, demo mode, or Google Sign-In. Once authenticated, users can access protected pages such as the dashboard and map.

Email, Demo, and Google Sign-In

The login system supports multiple entry paths. Email/password login provides a traditional account flow, demo mode allows quick testing without setup, and Google Sign-In supports a more realistic student login experience.



Milestone 02 authentication screen with email, password recovery, and Google Sign-In

On the backend, Atlas verifies credentials and issues a JWT. For Google Sign-In, the frontend obtains a Google ID token and sends it to the backend, where it is verified against the configured Google client ID. If verification succeeds, the backend returns the same application JWT format used by email login. This keeps the rest of the application independent of the authentication method.

JWT is used because it is a signed token format with a header, payload, and signature. In Atlas, the payload can carry basic claims such as the authenticated user's identity and expiry time, while the signature allows the backend to detect tampering. This fits the project because protected API routes can validate a request from the `Authorization` header without requiring the frontend to resend the password or requiring every API handler to implement its own login logic.

From a software engineering perspective, the design demonstrates separation of concerns and a clear security boundary. The frontend handles user interaction and stores the returned application token, while the backend performs trust-sensitive verification, password checking, Google ID token verification, and JWT signing. The main trade-off is external dependency: Google Sign-In requires OAuth configuration and network access to Google's verification service. Email/password and demo paths preserve an evaluable prototype when OAuth is unavail-

able.

Password Recovery and Change

M2 extends authentication with three endpoints: retrieving the configured security question, resetting a password after checking a normalised hashed answer, and changing a password after validating the current password from an authenticated request. Password hashing is kept in backend credential helpers rather than React components. This is still a prototype recovery mechanism—production deployment should prefer verified email reset tokens and rate limiting—but it exercises the complete account lifecycle without pretending a missing email service exists.

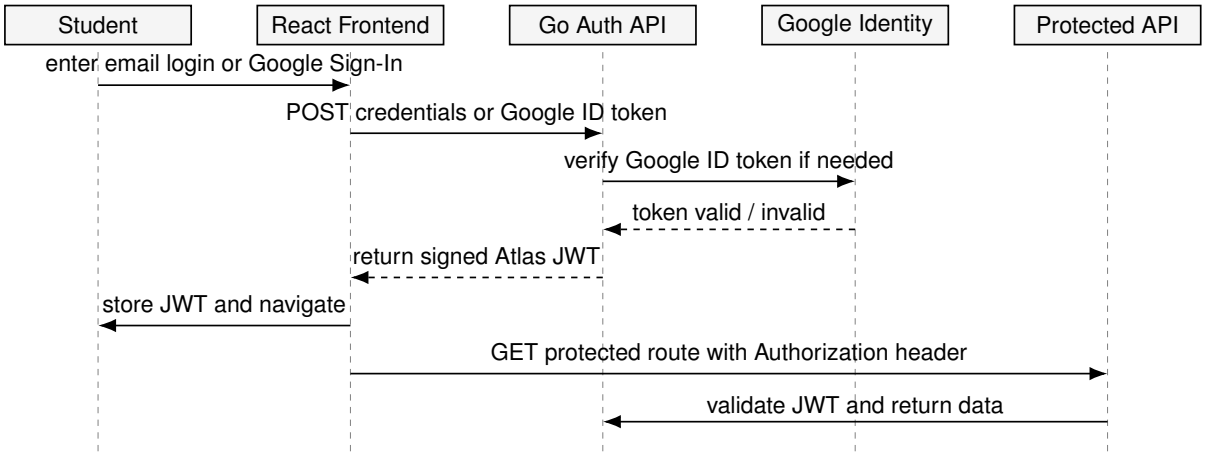
Protected Routes

Protected routes guard the dashboard and map pages. If a user is not authenticated, the frontend redirects them to the login screen. API requests also attach the JWT in the `Authorization` header, allowing the backend to reject unauthenticated access consistently.

This pattern reduces duplicated access checks. Instead of manually checking login state inside every page component, protected routing creates a reusable access boundary. It also makes later feature additions easier, because new protected pages can reuse the same wrapper.

Authentication Sequence Diagram

The sequence diagram below is placed in the authentication section because it explains the exact interaction order for email login, Google Sign-In, JWT issuing, and protected route access. The key design point is that the frontend may collect credentials or a Google ID token, but the backend remains the authority that verifies identity and issues the Atlas application JWT.



Sequence diagram for login, Google Sign-In, JWT issuing, and protected route access

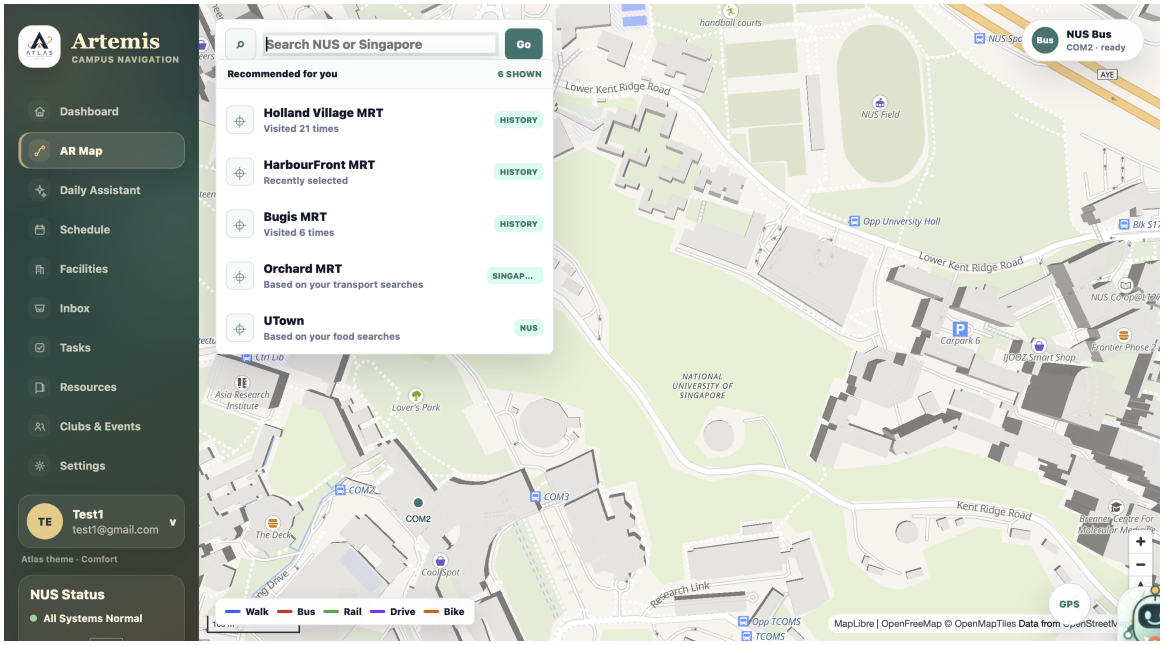
Search, Routing, and Map Rendering

The M2 navigation flow converts a user's intent into a route that can be reused by both the 2D map and AR view. It starts with local/external search, records a selected place, requests an itinerary for the chosen travel mode, normalises route alternatives, and draws the selected option.

Local-First Search and Ranking

The geocoding module contains curated NUS and Singapore places with aliases and tags. Queries are normalised and scored against names, descriptions, aliases, and tokens; results are merged with the external geocoder without duplicating places. A bounded browser history stores selected place summaries and increases the rank of repeated destinations. Time-of-day tags make study, food, shopping, or transport recommendations more relevant while keeping this lightweight personalisation on-device.

The fallback is deliberate: external geocoding may be slow or unavailable, but common campus abbreviations such as COM1 and UTown should still work. The UI begins with local matches and races the external request against a timeout, so the essential prototype answer is not blocked by one provider.

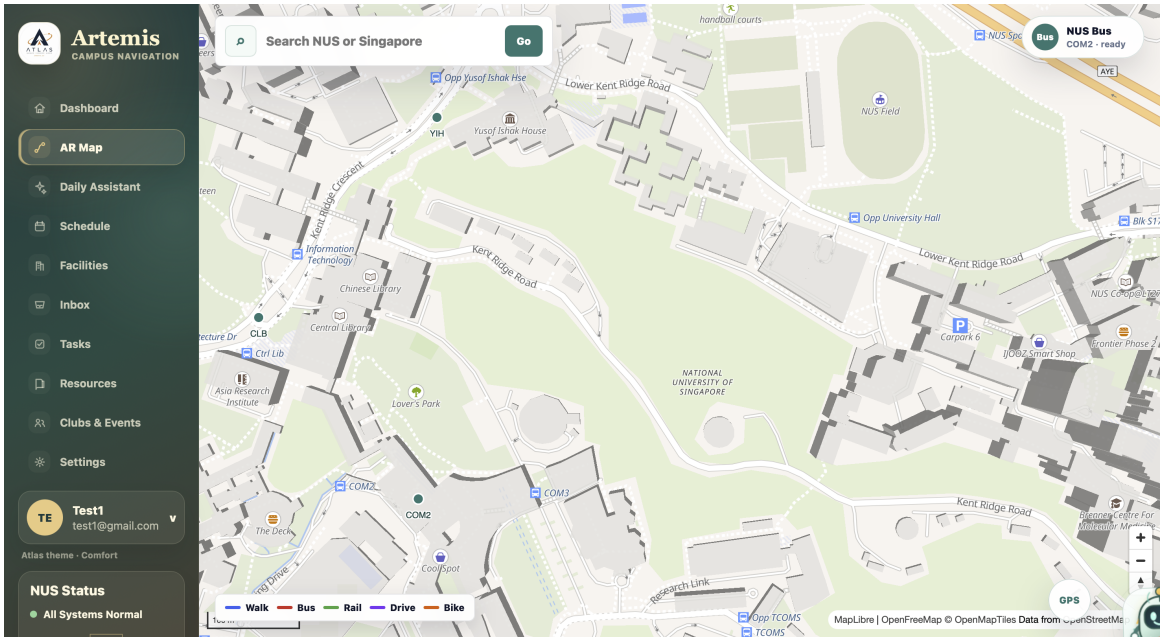


Contextual destination recommendations combining visit history, recent selections, transport interests, and NUS place tags

MapLibre and OSM Tiles

The frontend uses MapLibre to render map tiles and manage markers, route segments, and campus overlays. MapLibre is used because it is an open-source renderer that accepts Atlas-owned sources and layers instead of locking the project into a fixed map widget.

The map logic is separated into a map initialisation module, a `MapEngine` helper, routing/search services, a bus-layer module, and UI components. `MapEngine` encapsulates imperative MapLi-



Milestone 02 campus navigation map with NUS markers, search, route-mode legend, GPS, and shuttle status

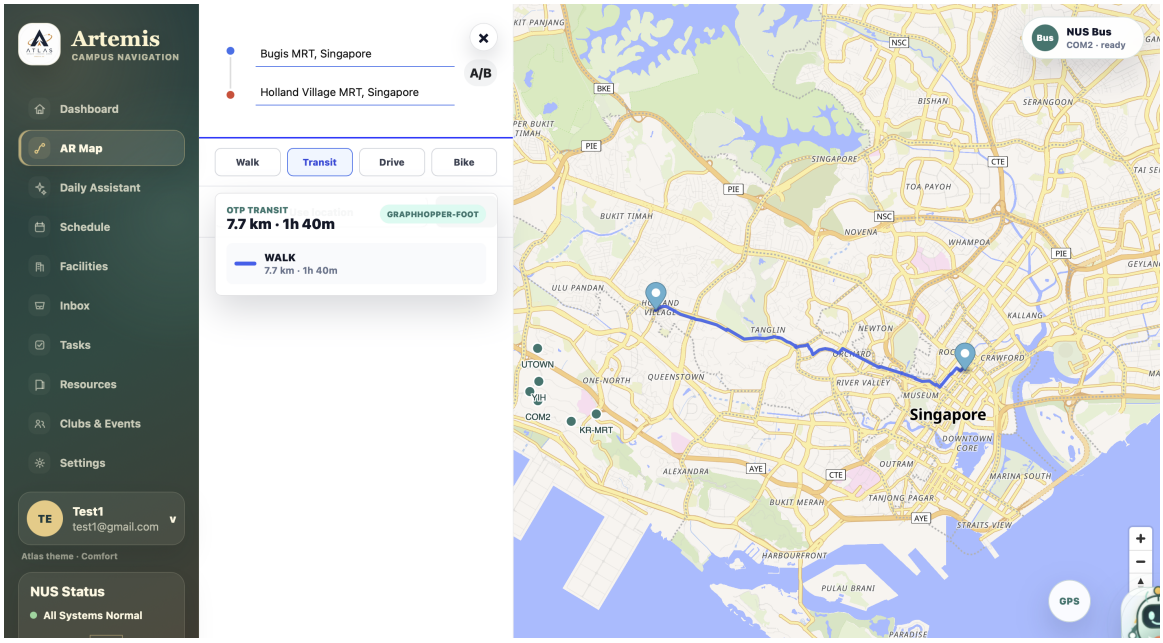
bre source/layer operations; React owns declarative state such as the chosen mode, destination, selected alternative, and loading/error state. This boundary avoids making React re-render the map object itself.

The bug tracked in issue #17 showed why this separation matters. The issue involved map building outlines and layer rendering. Such map bugs can come from coordinate format differences, layer ordering, or adding a layer before its data source is ready. By separating map engine actions from UI state, Atlas can debug and update rendering logic without changing authentication or dashboard code.

OpenTripPlanner Route Planning

The frontend sends `fromPlace`, `toPlace`, and mode to the protected `/api/otp/plan` endpoint. The Go adapter validates coordinates, sends a GraphQL query to OpenTripPlanner, converts legs into Atlas segments, and returns up to three alternatives. If a requested transit response has no drawable route, the server attempts a walking fallback and labels its source.

The client presents walking, cycling, driving, and transit choices and selects a display route. OTP remains the logical routing source; road-geometry refinement is isolated in display helpers. Sanitisation removes invalid coordinates and deduplicates points before drawing, because one malformed point can otherwise invalidate a complete MapLibre source.



Multimodal route result showing origin/destination, travel-mode selection, route source, distance, duration, and rendered geometry

Failure / Edge Case	Prototype Behaviour
Missing start or destination	Route request is not sent and the UI keeps the form active.
OTP not configured	Backend returns a service-unavailable error instead of a fabricated route.
Transit itinerary has no geometry	Backend attempts a walking fallback and identifies the fallback source.
Malformed/duplicate points	Client sanitises and deduplicates coordinates before drawing or opening AR.
External road refinement fails	The normalised OTP geometry remains the fallback display data.

Experimental AR Route Guidance

The AR view is an M2 prototype, not a claim of production-grade spatial navigation. When a drawable route is selected, the map creates an AR payload containing route label, travel mode, start/end places, distance, duration, segment modes, and coordinates. The payload is passed through router state and copied to session storage so a browser refresh does not immediately discard it.

Route Mathematics and Progress Tracking

`routeMath.js` normalises route points, computes haversine distance and cumulative progress, detects significant turns, converts geographic coordinates into a local metre frame, and finds the nearest point on every route segment. The nearest-segment result provides snapped

progress, off-route distance, segment index, and snapped coordinate. The UI treats a distance greater than 35 metres as off-route and derives the next manoeuvre from cumulative progress.

This computation is kept independent of Three.js and React state. The separation makes the geometric rules testable without a camera or WebGL renderer and prevents visual code from becoming the source of navigation truth.

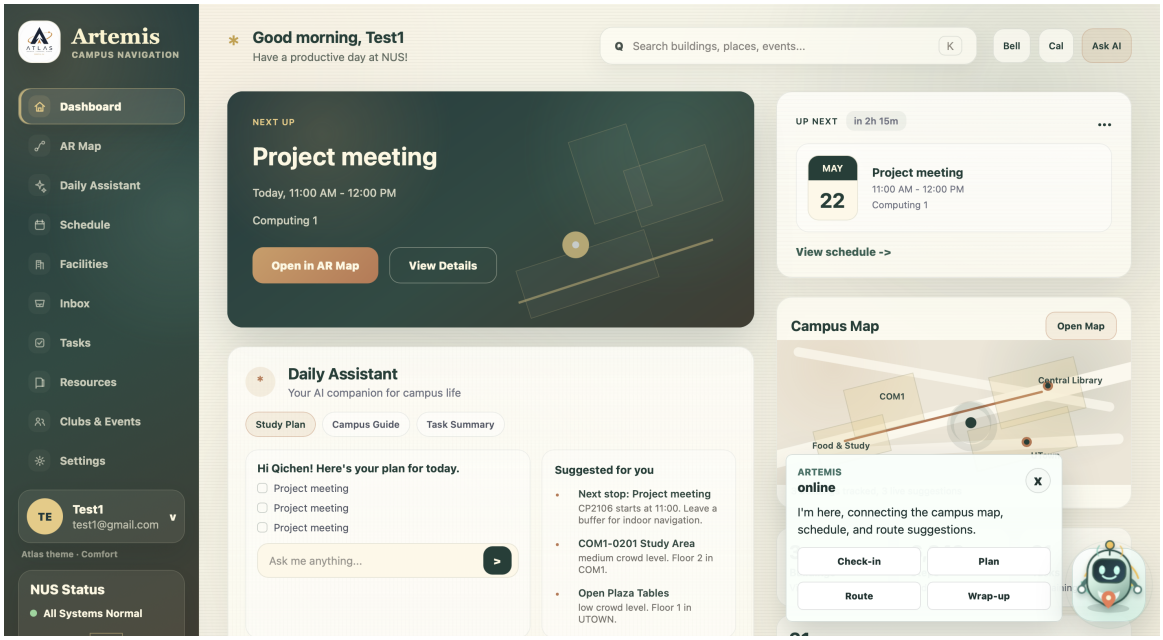
Three.js Camera Overlay

The AR screen layers a DOM video element behind a transparent Three.js canvas. It renders only a window around the user's progress rather than the complete route, then rotates the route group according to device heading. GPS updates come from `watchPosition`; compass input comes from device-orientation events; a calibration control aligns heading to the next route segment when sensor readings drift.

Permissions and sensor quality are the main constraints. Camera access requires a secure context, GPS accuracy can exceed the width of a campus footpath, and compass drift can rotate the overlay. The prototype therefore exposes camera/GPS status, heading, remaining distance, progress, and off-route distance instead of hiding uncertainty. M3 testing must be performed on real devices and must not rely only on desktop simulation.

Daily Assistant, Facilities, and Schedule

The daily assistant dashboard connects campus navigation to student routines. Instead of treating the map as a standalone feature, Atlas combines schedule items, next-location context, building data, facility filters, and recommendation cards into one student-facing workflow.



Milestone 02 dashboard combining the next event, schedule, map preview, recommendations, and assistant actions

Facility Discovery

Facility discovery focuses on practical indoor and near-indoor needs: study areas, lifts, restrooms, and printing locations. The backend seeds representative NUS building and facility data in SQLite and exposes it through protected API endpoints.

Endpoint / Data	Purpose
`GET /api/buildings`	Returns campus buildings with code, name, description, coordinates, floor count, and indoor support status.
`GET /api/facilities`	Returns facilities and supports filters such as `building=COM1` and `type=study_space`.
Seeded demo data	Allows evaluators to test building and facility discovery without requiring private production data.

The main design decision is to model facilities separately from buildings. This allows Atlas to add many facility records under one building and later attach richer indoor information such as floor plans, accessibility paths, or AR markers.

This design follows a simple data-normalisation principle: building information such as code, name, coordinates, and floor count should not be repeated inside every facility record. Facilities keep only facility-specific fields such as name, type, floor, crowd level, and the building

reference. This makes filtering easier and reduces the risk that one building is described inconsistently across multiple facility rows.

Schedule Management

Schedule management is included because a campus assistant becomes more useful when it understands where the student needs to go next. The current prototype supports viewing, creating, and deleting schedule items. Each item can include a location code, allowing recommendation cards to connect time-based context with campus data.

This feature also prepares the app for future NUSMods integration. Rather than hard-coding a static timetable into the frontend, the backend exposes schedule APIs and stores schedule data in SQLite. This gives the team a stable place to add timetable sync, conflict warnings, and route preparation in later milestones.

The engineering boundary is intentionally API-first. The frontend does not directly manipulate the database; it sends create/delete/read requests to backend schedule endpoints. This keeps validation and persistence in one place and allows future recommendation logic to reuse the same schedule data without duplicating state in separate frontend components.

Recommendation Cards

Recommendation cards summarise useful actions such as where the next class is, which building context matters now, or which facilities are likely relevant. These rules remain simple and explainable, which provides a stable fallback and comparison point for the new M2 assistant.

The deterministic recommendation endpoint is intentionally separate from the LLM agent. A generative response can fail or vary, while schedule/facility recommendations should remain available and predictable. This separation is a practical application of graceful degradation.

LLM Daily Assistant

The protected daily-assistant endpoint supports daily-plan, task-summary, and reflection modes. The backend builds five message groups: system role, mode-specific structured-output instruction, current time, serialised app context, and the user's request. A provider-neutral `LLMClient` interface means the agent depends on a small chat contract rather than one vendor SDK.

When the provider returns JSON, the parser extracts a human-readable reply plus optional schedule drafts; plain text is also accepted. Drafts are normalised before they reach the frontend. When no provider is configured or a provider fails, the endpoint returns a mode-specific deterministic fallback and a safe error category rather than leaking provider details. Four unit tests exercise successful structured output, unconfigured fallback, provider-error fallback, and plain-text parsing.

External Data and Transit Context

Atlas includes early integration points for external campus systems. These are important Artemis extensions because they move the project beyond a static campus map and toward a live assistant.

NUSMods Sync

The backend contains configuration for academic year, sync interval, HTTP timeout, and NUSMods integration. The dashboard exposes sync status and a manual sync trigger so users can see whether external data integration is healthy.

Sync status is modelled separately from schedule items and facilities because it describes the health of external data integration, not the data itself. This separation lets the dashboard tell users whether the app is using fresh, stale, failed, or manually seeded data without mixing operational metadata into user schedule records.

Endpoint / Setting	**Purpose**
`GET /api/sync/status`	Shows the latest sync state for campus or timetable data.
`POST /api/sync/run`	Allows a manual sync trigger during development and demonstration.
`NUSMODS_ACAD_YEAR`	Selects the academic year used by the NUSMods client.
`SYNC_INTERVAL`	Controls the scheduler interval for future automatic syncing.

Bus and Transit Context

Campus movement often involves a choice between walking, waiting for a shuttle, or combining both. Atlas therefore includes bus context as an Artemis extension feature. The backend supports endpoints for bus stops, routes, arrivals, active vehicles, alerts, and context near a latitude/longitude point.

The current design keeps bus data server-side. This avoids exposing private API keys in the browser and allows the backend to cache, normalise, or fall back to demo data. Each response carries source information so demo data is not presented as live telemetry.

The bus module uses the same engineering principle as other external integrations: the frontend consumes Atlas backend endpoints, while the backend handles third-party API credentials, response normalisation, caching, and fallback behavior. This makes the user-facing dashboard more stable because it does not need to understand every raw bus API response shape.

Bus API	Use Case
`GET /api/bus/stops`	List campus shuttle stops.
`GET /api/bus/routes`	List available NUS shuttle routes.
`GET /api/bus/arrivals`	Show estimated arrivals for the requested stop code.
`GET /api/bus/active`	Show active vehicles for the requested route.
`GET /api/bus/alerts`	Show service alerts returned by the shuttle provider.
`GET /api/bus/context`	Provide nearby transit context for a latitude/longitude pair.

Singapore Public Transport

The Singapore transit adapter extends the same proxy pattern to LTA DataMall. It provides bus-stop data, nearby-stop queries calculated from latitude/longitude, arrival estimates for a selected stop, and rail service alerts. The account key is read only by the Go server. When it is absent, the API returns an explicit configuration error rather than embedding credentials or silently returning invented live values.

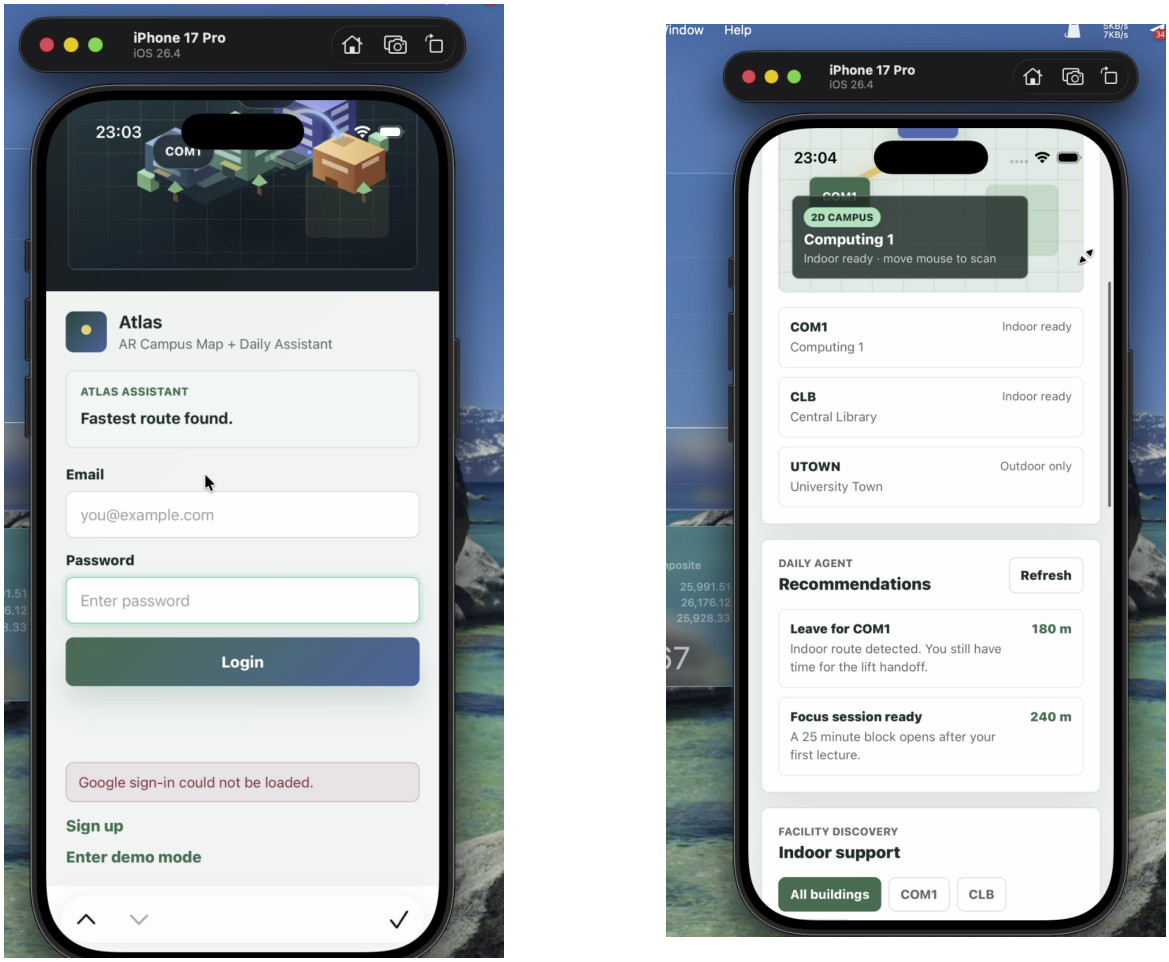
This integration does not claim live MRT arrival times because the selected LTA source does not provide them. Atlas limits its rail feature to service alerts and documents that boundary. This is an example of designing to the actual provider contract instead of expanding the feature claim beyond the data available.

Mobile Frontend Experience

Because campus navigation is often used while students are moving, mobile usability is a supporting but important feature. Issue #10 tracks responsive layout and manual viewport checks for authentication, dashboard, map, schedule, and recommendations.

Responsive Layout

The frontend is designed to remain readable on common mobile widths. Cards, buttons, filters, and map panels are arranged to avoid horizontal scrolling and text overlap.



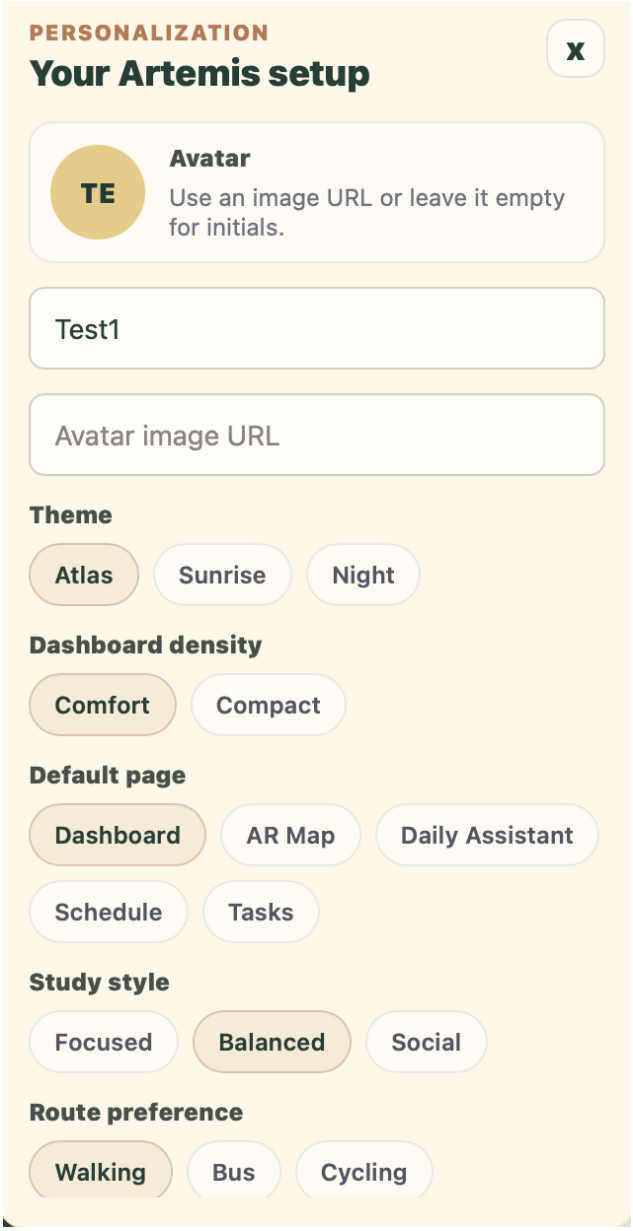
Mobile App Screens

The main engineering principle here is progressive enhancement: the application should remain usable on desktop while also adapting to mobile. The trade-off is extra CSS and layout testing, but this matters for a campus assistant because students are more likely to use it on phones than on large desktop screens.

Personalisation Settings

The personalisation panel lets users choose an avatar, visual theme, dashboard density, default landing page, study style, and preferred travel mode. These preferences are kept separate

from route computation: presentation choices affect how Atlas opens and displays information, while travel preferences provide context that routing and recommendation modules may consume explicitly. This avoids coupling visual theme state to navigation correctness.



Mobile personalisation panel for avatar, theme, dashboard density, default page, study style, and route preference

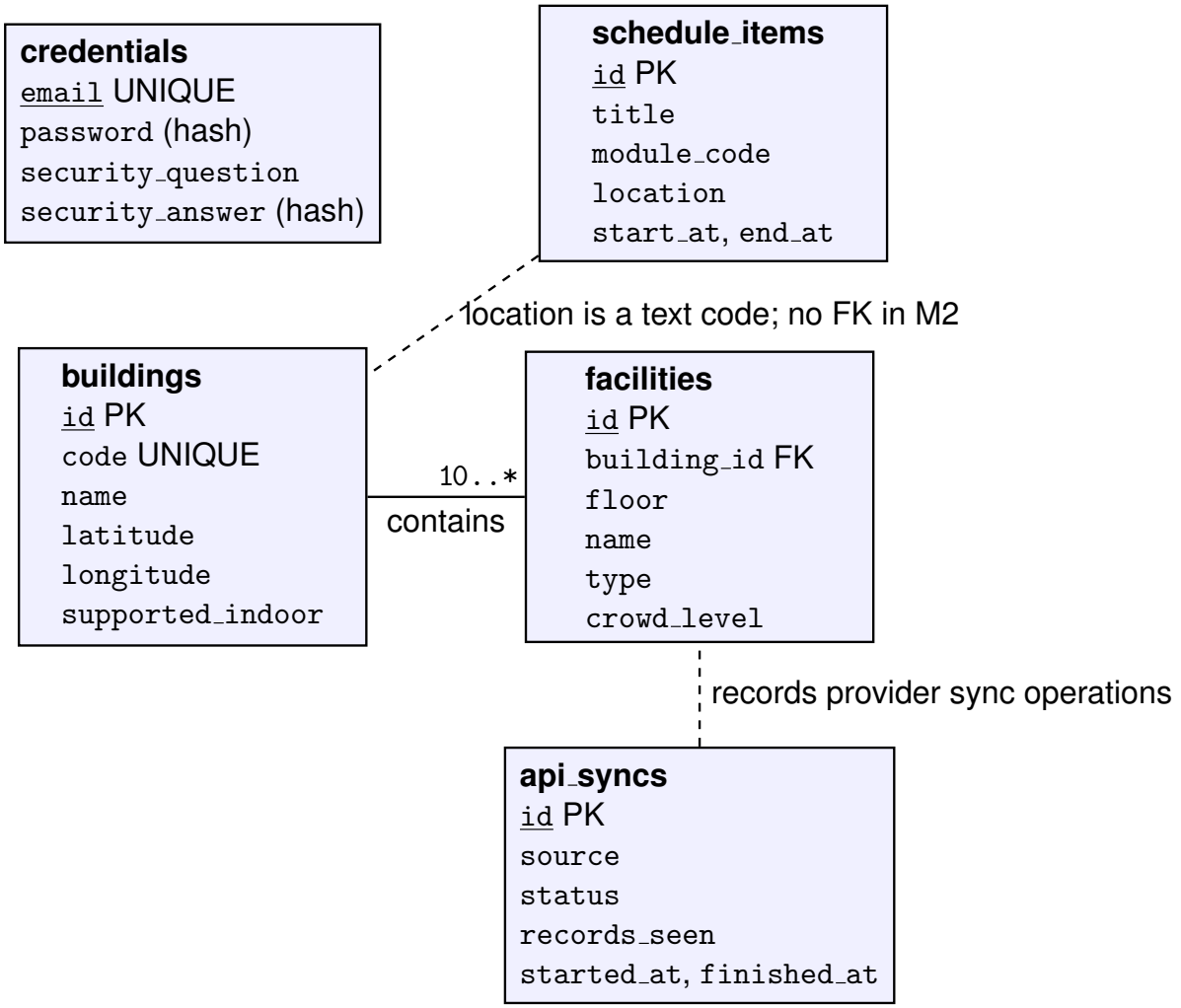
Standardised Software Diagrams

In addition to the high-level architecture diagram and the authentication sequence diagram, the following standardised diagrams describe persistent data relationships and the map/search user journey.

ER Diagram

The ER diagram models the schema actually created by `Store.Migrate`. The notation uses primary keys (PK), foreign keys (FK), and relationship cardinalities. Buildings contain facilities; schedule items and API sync records are currently prototype-wide rather than user-owned.

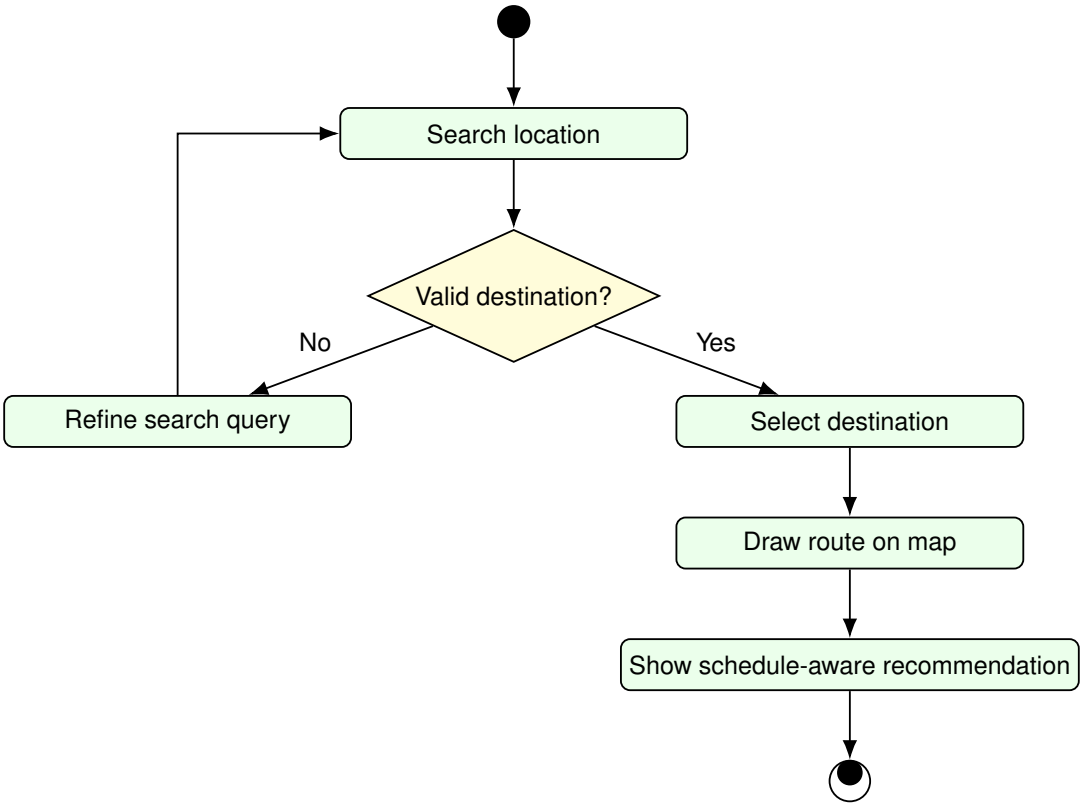
The database reduces duplication by storing building data once and linking facilities through `building_id`. Credentials are separated from campus content. A known M2 limitation is that `schedule_items` has no `user_id`; every authenticated user currently reads the same prototype schedule. Adding an account key and migration is required before the app stores real personal timetables.



M2 SQLite schema and its current prototype-wide schedule limitation

Activity Diagram

The activity diagram describes the user journey for map search and route-oriented recommendations. The flow starts from a location search, moves through destination selection and route rendering, and ends with contextual recommendations.



Activity diagram for search location, destination selection, route drawing, and recommendation display

Automated Testing and QA

M2 explicitly requires testing from unit to system level. Atlas currently has genuine backend unit automation, build-level integration checks, and a manual system matrix. The repository does **not** yet contain frontend unit tests or automated browser end-to-end tests; this report states that gap rather than presenting designed cases as executed automation.

Automated Checks

Level / Check	Command	Observed M2 Result and Scope
Unit: assistant agent	`cd backend && go test ./...`	PASS. Four tests cover structured LLM output, missing configuration, provider failure, and plain-text parsing. All Go packages compile.
Integration/build: frontend	`cd frontend && npm run build`	PASS. Vite transformed 355 modules and produced a production bundle. It also reported a large-chunk warning, recorded below as a performance risk.
Deployment definition	`docker compose config` and repository review	Dockerfile, app/OTP/Caddy services, volumes, and environment boundaries are defined. A full image rebuild is an environment-dependent release check, not counted as a unit test.
Document build	`latexmk -pdf readme.tex`	PASS after M2 updates; confirms all diagrams, tables, links, and images resolve into the final PDF.

Executed Unit Test Design

The assistant tests use dependency injection: `DailyAssistantAgent` receives a fake implementation of the small LLM interface. This isolates prompt construction and parsing from network cost, rate limits, and provider variability.

Automated Test	Technique	Assertion
`ReturnsLLMReply`	Valid structured equivalence class	Reply, normalised schedule draft, provider/model metadata, and all five message groups are present.
Unconfigured-provider fallback	Missing-dependency/error guessing	Provider is not called; response is marked unsuccessful and uses the daily-assistant fallback.
Provider-error fallback	External failure injection	A provider error produces a safe category and reflection fallback rather than leaking the raw error.
Plain-text parsing	Alternate response-format class	Plain text remains a valid reply and does not create unintended schedule drafts.

High-value next unit targets are `routeMath.js` boundary cases (invalid coordinates, duplicate points, first/last segment, 35-metre off-route threshold), search scoring, OTP response normalisation, authentication middleware, and schedule ownership after the schema is corrected.

Manual Test Matrix

System Flow	Procedure	Acceptance Criterion
Demo login	Use seeded demo account `test1@gmail.com` / `cp2106`.	User reaches the protected dashboard and receives an application JWT.
Registration	Register a new account through the auth screen.	New credentials are accepted and can be used to access protected pages.
Google Sign-In	Sign in with a configured Google OAuth client.	Backend verifies the ID token and returns the same app JWT format.
Dashboard loading	Open protected dashboard after login.	Health, buildings, schedule, recommendations, facilities, and sync status appear.
Facility filtering	Change building and type filters.	Facility list updates without breaking dashboard layout.
Schedule actions	Add and delete schedule items.	Schedule list updates and recommendation context remains consistent.
Campus map	Open map page, search location, and draw route.	Map loads, marker/route interactions work, and the UI remains usable.
Multimodal route	Choose start/end and switch walking, cycling, driving, and transit modes.	A supported itinerary or explicit provider/configuration error appears; alternatives and segments remain selectable.
AR hand-off	Draw a valid route, open AR guidance, deny/allow sensor permissions as appropriate.	The same route loads, status remains visible, and denial does not crash the page. Real-device accuracy is a pending M3 test.
NUS/LTA transit	Select a shuttle route/stop and query Singapore transit where credentials are configured.	Source-labelled arrivals/alerts appear; missing credentials or network failure is explicit.
Assistant failure	Run each assistant mode with the LLM configured and unconfigured.	A structured/provider reply or deterministic fallback is returned without blocking the dashboard.
Mobile viewport	Test auth, dashboard, map, and mobile screenshots at narrow widths.	Text, cards, buttons, and map panels remain readable without horizontal scrolling.
Production app	Visit the Render deployment URL.	The deployed app loads over HTTPS and supports the same demonstration flow.

Build, Deployment, and CI Gap

The repository has a reproducible build/deployment path, but it does not contain a committed `.github/workflows` file at the M2 cut-off. Therefore Atlas does not claim automated continuous integration yet. This is below the desired Artemis practice and is a concrete M3 corrective action.

Required CI Gates

The following gates are the minimum workflow to add for pull requests and pushes to `main`. Until the workflow is committed and visible in GitHub checks, they are manually executed release gates.

Pipeline Stage	Command / Action	Why This Stage Is Included
Checkout	Fetch repository source	Ensures CI tests the same version of frontend, backend, deployment, and documentation files that will be reviewed or merged.
Frontend dependency install	`npm ci`	Uses the lockfile to install repeatable dependencies, reducing "works on my machine" differences between teammates and CI.
Frontend build	`npm run build`	Catches React/Vite import errors, broken assets, and production bundling failures before deployment.
Backend tests	`go test ./...`	Checks Go package compilation and backend test coverage for API, auth, storage, scheduler, and integration helpers.
Docker build	`docker build .`	Confirms the app can be packaged in the same production-style environment used for deployment.
Deployment validation	Manual Render walkthrough	Confirms the deployed HTTPS app supports the actual demo flow, including auth, dashboard, map, schedule, and bus contexts.

The workflow should fail fast on unit/build errors, cache only immutable dependency data, and make frontend, backend, and container results independently visible. A deployment step should run only after protected-branch review and successful gates; it should never expose repository secrets to pull requests from untrusted forks.

Continuous Deployment Path

Atlas currently uses a production deployment path built from the repository's `Dockerfile`, `docker-compose.yml`, Caddy configuration, and Render-hosted application URL. The deploy-

ment flow is:

1. Merge reviewed changes into `main`.
2. Build the frontend with Vite.
3. Build the Go backend binary.
4. Package the production app through the multi-stage Dockerfile.
5. Serve the app over HTTPS through the configured deployment host.
6. Validate the deployed app using the manual test matrix and video walkthrough flow.

Environment variables such as `JWT_SECRET`, `GOOGLE_CLIENT_ID`, `DB_PATH`, `ALLOWED_ORIGIN`, and bus API credentials are kept outside source code. This keeps deployment configurable while avoiding accidental exposure of private credentials.

Software Design Principles

The application was developed using established software design principles to improve maintainability, readability, reusability, and scalability. Table 3 summarizes how key features of the system apply these principles.

Table 3: Application of Software Design Principles

Feature	**Design Principle**	**Description**
Place-search service	Single Responsibility, Abstraction	Handles only the retrieval of location suggestions from the geocoding API, while hiding API implementation details from other components.
`SelectList`	Modularity, Reusability	Provides a reusable component for displaying selectable suggestions independently of the data source.
`RoutingForm`	Separation of Concerns, Modularity	Manages user inputs and routing interactions while delegating suggestion display and map functionality to dedicated components.
`MapEngine`	Encapsulation, Single Responsibility	Centralises all map-related operations, such as marker placement, route drawing, and map navigation, within a single class.
`drawRoute()`	Abstraction	Provides a simple interface for route rendering without exposing underlying MapLibre source and layer management details.
Location tracking methods	Encapsulation	Encapsulate location marker creation and updates, hiding implementation details from the rest of the application.
Transit provider adapters	Adapter, Information Hiding	Translate provider-specific authentication and payloads behind stable Atlas endpoints so credentials and schema changes do not spread into the frontend.
Daily assistant and LLM interface	Dependency Inversion, Strategy	The agent depends on a small chat interface; configured providers and test fakes can be substituted without changing orchestration logic.
`routeMath.js`	Pure Functions, Separation of Concerns	Keeps route geometry, progress, bearing, and manoeuvre derivation independent from Three.js rendering and React lifecycle state.
`otpPlan` normalisation	Anti-Corruption Layer	Converts OpenTripPlanner GraphQL structures into Atlas route segments and alternatives, protecting the client from provider-specific nesting.

Milestone 02 Progress and Honest Limitations

Milestone 02 turns the M1 full-stack foundation into a broader navigation prototype. The current repository contains a React/MapLibre/Three.js client, Go API and provider adapters, SQLite persistence, account lifecycle, dashboard context, campus and Singapore search, multimodal OTP routing, NUS/LTA transit endpoints, an LLM assistant with deterministic fallbacks, an experimental AR screen, and a Docker/Render deployment path.

The most important end-to-end prototype path is present: authenticate, select a start and destination, choose a mode, inspect a route on the map, and pass that exact route to AR guidance. The prototype also exposes uncertainty and configuration failures instead of labelling every result as live.

The following limitations materially affect Artemis readiness:

- Frontend route mathematics, search ranking, components, and full user journeys do not yet have automated tests. Current automation is concentrated in four backend assistant tests and build checks.
- No GitHub Actions workflow is committed, so the documented gates are not automatically enforced on pull requests.
- Schedule records are not linked to a user account in the M2 database schema; personal schedule isolation must be added before real use.
- AR is a browser-based visual prototype. It has not yet demonstrated robust alignment under real campus GPS/compass noise or indoor conditions.
- Indoor floor plans, room-level paths, accessibility constraints, and verified emergency data are not implemented. User stories covering them remain product scope, not M2 completion claims.
- External providers can be unavailable, rate-limited, credential-gated, or change schema. Fallbacks improve evaluability but do not replace integration monitoring.
- The frontend production build succeeds but emits a large JavaScript chunk warning; Three.js, mapping, and dashboard code should be split by route before production use.

Planned Next Steps

- Add Vitest tests for route mathematics/search and component tests for auth/map state; add Playwright system flows and API handler/store integration tests.
- Commit GitHub Actions gates for `npm ci`, frontend build/tests, `go test ./...`, and Docker build, then require them before merge.
- Migrate schedules to explicit user ownership and add authorization tests preventing cross-user access.
- Conduct task-based tests with NUS students on real phones, measuring route completion, wrong turns, time-to-understand, GPS error, and perceived AR usefulness.
- Improve AR calibration/off-route recovery and introduce route-based code splitting to reduce initial bundle size.

- Add verified indoor/accessibility data only for a small pilot building set, then expand based on observed navigation failures.

Evaluator Demo Route

The recommended M2 walkthrough focuses on one coherent user answer rather than visiting every screen independently:

1. Sign in with the documented demo account and confirm that a protected dashboard loads.
2. Review the next schedule item and deterministic/LLM assistant context.
3. Open the map, use current location or a campus start, and search for an NUS destination.
4. Compare route modes and alternatives; inspect distance, duration, and coloured segments.
5. Open the AR view from the selected route and inspect progress, heading, camera/GPS status, next manoeuvre, and off-route feedback. Sensor permissions may be denied safely for a desktop demonstration.
6. Return to the map and inspect NUS shuttle or Singapore transit context, paying attention to the displayed provider/fallback source.

This route demonstrates the core M2 claim: Atlas connects daily context, search, routing, transport, and an experimental AR presentation through explicit module and service boundaries. The limitations above define the work still required before the prototype should be treated as a reliable campus navigation product.